Supplemental Source Surface

Supplemental Source Surface I

This supplement records the top-level source surfaces a reader or reviewer should inspect before turning any prose in the main manuscript into a verifiable claim. Each surface below is authored material under version control; the generated artifacts it produces are described separately so that the boundary between hand-written source and reproducible output stays explicit.

The `src/` tree is the executable core of GNN. It is organized into 31 Python packages spanning 582 source files and roughly 194808 lines of code. Alongside these packages sit the 25 top-level step modules (0–24) that implement the numbered pipeline as thin orchestrators delegating into the packages, each module owning one stage of the progression from a parsed GNN text model through visualization, type checking, code export, and executable cognitive simulation. Every claim the manuscript makes about pipeline behavior should be traceable

to one of these step modules rather than to descriptive prose alone.

The `input/` tree holds the model corpora that exercise the pipeline. Its `gnn_files/` subtree contains 10 corpus directories organized by model family, together totaling 29 example files. Outside that subtree, `input/multi_agent_models/` holds 1 model file and is registered by no manifest family; `input/recursive_models/` holds no model files and is registered by no manifest family. That model file is outside the 29-file count above. A `model_family_manifest.json` enumerates the 9 registered families and their target directories. This manifest is the authoritative registry that downstream steps and the manuscript variable producer read when they report family counts and cross-framework coverage; it should be consulted directly rather than inferred from directory listings.

Supplemental Source Surface I

The `scripts/` tree contains the thin orchestrators that gate and reproduce the project. These include the acceptance scripts that confirm the pipeline runs end to end, the reliability gates that enforce determinism and coverage expectations, and the manuscript variable producer (`src/manuscript_variables.py` driven from this layer) that emits the double-brace `{{...}}` token values consumed throughout the manuscript. Treating these scripts as the source of reproduction commands keeps reported numbers bound to what the code actually computes.

Supplemental Source Surface I

The `doc/` tree is the prose and reference surface, comprising 616 files of specification, tutorial, and design documentation for the GNN language and its Active Inference grounding (Smékal and Friedman 2023). It is the place to verify that a manuscript statement about GNN syntax or semantics matches the documented language rather than a convenient paraphrase.

The `output/` tree collects per-step pipeline artifacts: the data dumps, intermediate representations, validation reports, and the 105 figure artifacts committed under it. Everything here is disposable and reproducible from the surfaces above, so it should be read as evidence of a run rather than as authored source. Notably, `output/data/manuscript_variables.json` is where the manuscript's substituted token values are materialized.

Authored Source Versus Generated Output I

`src/`, `scripts/`, `input/`, `doc/` and `manuscript/` are authored and reviewed; everything under `output/` is generated and disposable, including `output/manuscript/`, which holds the token-substituted copies of the `manuscript/` sources rather than the sources themselves. The commands that regenerate the output surfaces are listed in sec. 6: the smoke run for the per-step artifacts, `scripts/z_generate_manuscript_variables.py` for the token map and the hydrated sections, `scripts.manuscript_build_figures` for the 6 manuscript figures, and the template render stage for the PDF. Which values become tokens is not a matter of taste: any quantity the producer can derive from a source surface is a token, `scripts/check_manuscript_tokens.py` fails the build when a

section hard-codes one instead, and the resulting 86-entry map is committed at `output/data/manuscript_variables.json` for audit. External references are resolved from `manuscript/references.bib`, which the same gate cross-checks against every citation key in the prose. This manuscript quotes no private or unpublished material.